

Thanos the Hero

Day 2 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“population must be strictly greater”*
- ▶ *“minimize the number of lives he has to take”*

Programming interpretation:

- ▶ The preferred order imposes a strict increasing condition in the input direction.
- ▶ It is easier to enforce by scanning backward from the least preferred world.
- ▶ Whenever a population is too large, reduce it just enough to restore strictness.

Textbook connection: CP4 3.2 i. Mathematical Simulation, Harder

Core technique: Backward greedy simulation

- ▶ Planet populations must become strictly increasing from left to right in the original order.
- ▶ We may only reduce populations.
- ▶ The goal is to remove the fewest people while making the condition hold.

- ▶ Working from the end fixes the maximum allowed value for the previous planet.
- ▶ If a population is too large, lowering it just enough is always best.
- ▶ If a value must become negative, the task is impossible.

- ▶ Reverse the populations so the required relation is checked left to right.
- ▶ For each next population, ensure it is strictly smaller than the previous reversed value.
- ▶ If it is not, subtract the minimum necessary difference.
- ▶ Add the removed amount to the death toll.
- ▶ If any adjusted population becomes negative, print 1 and stop; otherwise print the total.

- ▶ For adjacent reversed values a then b , we need $b < a$.
- ▶ When $b \geq a$, the smallest valid replacement is $a - 1$.
- ▶ Reducing more would never help earlier planets and would only increase the cost.

- ▶ The suffix already processed is fixed to a feasible minimum-cost form.
- ▶ At each step, choosing the largest allowed value for the current planet minimizes removals and leaves maximum room for earlier planets.
- ▶ If that largest allowed value is negative, no nonnegative population can satisfy the strict order.

Complexity: $O(N)$ time and $O(1)$ extra memory apart from the input list.

- ▶ The implementation mutates the reversed list in place.
- ▶ Watch the exact output for impossible cases.

Source lines: `thanosthehero.py:27-32`

```
import sys

lines = sys.stdin.read().strip().splitlines()
cases = int(lines[0])
pops = list(map(int, lines[1].split()))[::-1]
deathToll = 0
```

What this chunk does

- ▶ The input order is reversed so the scan moves from lower preference to higher preference.
- ▶ `deathToll` accumulates the minimum number of people removed.
- ▶ The unused `cases` variable mirrors the first input value.

Source lines: `thanosthehero.py:34-42`

```
for i in range(1, len(pops)):
    if pops[i - 1] <= pops[i]:
        diff = (pops[i] - pops[i - 1]) + 1
        deathToll += diff
        pops[i] -= diff
        if pops[i] < 0:
            print(1)
            quit()
print(deathToll)
```

What this chunk does

- ▶ If the next preferred world is not strictly larger, reduce it to one less than the previous world.
- ▶ `diff` is exactly the smallest reduction that restores the inequality.
- ▶ If a population becomes negative, the target universe is impossible, so the required output is 1.

Solution file: `practice_contests/day_2/thanosthehero.py`

Problem: <https://open.kattis.com/problems/thanosthehero>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.